

OSDU SPI

The Engineering System Behind Azure's OSDU Service Forks

Source material compiled from the Azure/osdu-spi repository

September 2026

Contents

About This Document	5
Part I: The Problem	6
What OSDU Is	6
The Provider Model, and the Seam Called SPI	6
Three Things Called SPI	7
Why a Fork, and Why It Cannot Be a Snapshot	7
The Deletion That Forces the Design	8
What This System Is	8
What This System Is Not	9
Part II: The Shape of the System	10
One Template, Many Forks	10
Two Contexts in One Repository	10
Three Branches, Three Different Jobs	11
Ownership Runs Through the Tree	12
Two Tiers	12
Part III: Becoming a Fork	13
Initialization Is a Conversation	13
Split What Fails Differently	13
The Bootstrap Problem	13
The One-Time Seed	14
Adoption, the Other Door	14
Settings as Code	15
Part IV: Generate the Branch, Do Not Merge Into It	16
The Conflict That Kept Coming Back	16
The Reframe	16
Building a Tree with Plumbing	17
What the Filter Removes	17
Halt on the Unknown	18
Provenance in Two Trailers	18
Proving the Filter Before Trusting It	19
What the Fork Now Owns Forever	19

Part V: The Cadence	20
A Sync That Remembers	20
The Meta Commit	21
The Cascade	22
Labels Are the State Machine	22
Human-Triggered Steps Need a Sweeper	23
Cleaning Up After Itself	23
Releases, and the Tag That Says Which Upstream	23
Part VI: Building Someone Else's Code	25
Java 17, Maven, and a Repository You Do Not Own	25
The Profile Trap	25
The Dockerfile Nobody Maintained	26
A Public Registry, and a Setting That Cannot Be Fixed	27
Coverage Without a Threshold	27
Part VII: Trust	28
Why the Default Token Is Not Enough	28
Know the Event Context Before Building a Workaround	28
The Guard Clause Is the Policy	29
A Required Check That Always Runs	29
Dependabot, Split by Ownership	30
Part VIII: Proving It Against a Real Environment	31
The Question Every Test Asks First	31
Five Constraints That Rule Out the Obvious Answer	31
Three Contracts, Three Owners	32
The Descriptor	32
What the Repository Is Allowed to Know	33
Borrow, Prove, Restore	34
Secretless Callers	36
Tests as Images	36
Answers That Must Be Created	37
What Is Not Yet Wired	37
Part IX: The Second Tier	39
Why It Must Be a True Fork	39
Mirror Mode	39
What Changes, and What Does Not	41
The Round Trip	41

What the Second Tier Costs.....	42
Part X: The Things That Are Not True	43
Part XI: Writing Down What Was Learned	45
A Register Organized by Question	45
A Second Document, Derived Rather Than Authoritative.....	45
The Most Useful Entry Is About a Failure	46
The Habit Behind It.....	46
Glossary	48
Appendix A: The Decision Register	51
Appendix B: Workflow Reference	54
The template's own workflows	54
Workflows deployed to every fork	54
Schedules.....	55
Labels	55
Sources	56

About This Document

This is a single, self-contained explanation of the OSDU SPI engineering system: the problem it solves, the shape it takes, the mechanisms it runs on, and the reasoning behind the choices its authors made.

It is written for a technically literate reader who is not assumed to already know OSDU, Git forking strategy, GitHub Actions, or Maven. Each of those is introduced where it first matters.

The material is drawn from the project's own documentation: an architecture overview, five design guides, seven workflow references, two runbooks, a register of thirty-eight architectural decision records, and a companion file of learnings written in the first person plural. Where this document describes a trade-off, the trade-off is the project's own stated reasoning, not an outside assessment. Where it cites an exact string — a branch name, a label, an exit code, a cron expression — that string was read from the implementation, not from the prose about it.

A note on subject matter. This system is not the thing that deploys OSDU to Azure. That is a separate project, and this document says so explicitly in Part I, because the two share a name and are easy to confuse. This document is about the machinery that produces the code in the first place: how a permanent fork of somebody else's actively developed codebase stays current, stays buildable, stays releasable, and stays honest, across many repositories, indefinitely.

A note on honesty. This project documents its own failures with unusual directness. It records a feature that crashed on every single invocation for months while reporting success. It records a state flag that vouched for an environment that no longer existed. It records which gates are designed but not yet wired. Those admissions are preserved here rather than smoothed over, because they are frequently the most instructive parts of the story.

Part I: The Problem

What OSDU Is

The energy industry has a data problem that is fundamentally a translation problem.

A single operator might hold seismic surveys, well logs, drilling records, production measurements, and geological interpretations, gathered over decades, stored in dozens of incompatible systems, produced by competing vendors who each had their own idea of what a “well” is. Merging two companies means merging two of these archives. Adopting a new analysis tool means writing yet another adapter.

OSDU — the Open Subsurface Data Universe — is the industry’s collective answer. Developed under the OSDU Forum, it is both an open standard and an open-source reference implementation defining a common data platform for subsurface and energy data. It specifies a set of services with agreed-upon APIs: a storage service holding records, a schema service defining what those records may look like, a search and indexer pair making them findable, a legal service tracking compliance tags, an entitlements service answering authorization questions, a partition service keeping tenants separate, and others for files, workflows, and unit conversion.

The reference implementation lives in GitLab, at the Open Group’s community instance. It is large, it is actively developed by many contributors across several companies, and it never stops moving.

The Provider Model, and the Seam Called SPI

OSDU was designed to run on any cloud. That ambition creates a structural requirement.

Every one of those services needs somewhere to actually store things. Records need a document database, entitlements needs a graph, files need object storage, search needs a search engine, and inter-service notification needs a message bus. OSDU’s answer is a provider model: the business logic of each service is shared, and the cloud-specific plumbing is pushed behind swappable implementations.

In the OSDU source tree this appears as a directory convention. A service named `partition` has a `partition-core` module holding the shared logic, and then `provider/partition-azure`, `provider/partition-aws`, `provider/partition-gc`, and others — one set of rules, several sets of plumbing. The interface between them is a *service provider interface*, and that seam is what gives this project its name.

Three Things Called SPI

The abbreviation is used for three different things in this ecosystem, and confusing them makes everything harder to follow. They are distinguished here once.

The interface. The original meaning: OSDU’s own pluggability seam, the boundary between shared service logic and a cloud-specific implementation. This is a property of OSDU’s source code and belongs to the community.

The Stack. A separate Azure project, `osdu-spi-stack`, which provisions Azure infrastructure and deploys a complete running OSDU environment onto it. It answers the question “how do I get a working OSDU on Azure.”

The engineering system. `osdu-spi` — the subject of this document. It answers a different question: “where does the Azure code come from, and how does it stay current with a community project that is still moving.” It produces the service repositories and the container images that the Stack deploys.

The two Azure projects meet at exactly one seam, described in Part VIII: the engineering system builds an image, borrows a slot in a running Stack environment to prove it works, and gives the slot back. Everywhere else they are independent.

Why a Fork, and Why It Cannot Be a Snapshot

Azure needs its own provider implementation, developed on its own schedule, held to its own review and security standards, and released as artifacts Azure can support. That requires a repository Azure controls. The obvious mechanism is a fork.

The difficulty is that a fork is usually understood as a snapshot: you copy a codebase, you change it, and the two drift apart. That model fails here in both directions.

If the fork stops taking upstream changes, it stops being OSDU. Compatibility with the community standard is the entire value proposition; a divergent implementation that no longer tracks the specification is a proprietary archive with extra steps.

If the fork takes upstream changes by hand, the cost compounds. The project’s own framing names four failure modes. Integration is slow, and slowest precisely when an upstream change touches an interface the Azure implementation depends on. Local modifications drift, so each synchronization is harder than the last. In a shared tree a build failure in any one provider’s implementation blocks every other provider. And without deliberate tracking, nobody can say which upstream release a given fork version actually contains.

The system’s own comparison is blunt about the delta it targets:

Aspect	Manual fork management	This system
Synchronization	Weekly or monthly, by hand	Daily, as a reviewable pull request

Aspect	Manual fork management	This system
Conflict resolution	Wherever the merge happened	Isolated in a dedicated branch
Release coordination	Tracked by hand, if at all	Correlation tags against upstream releases
Integration testing	After conflicts are resolved	At each branch stage

The fork, in other words, must be permanent and continuously integrated. Not a copy. A relationship.

The Deletion That Forces the Design

There is one more constraint, and it is the one that gives the architecture its particular shape.

Upstream intends to remove its Azure provider implementations.

This inverts the usual fork problem. Normally a fork worries about upstream changing code the fork depends on. Here the fork must contend with upstream *deleting* code the fork depends on, and deleting it permanently. After that deletion, `provider/partition-azure` will not exist upstream at all.

The consequence is that the fork cannot merely track upstream. It must take permanent ownership of a specific subtree, seeded once from the newest upstream revision that still contains it, and then maintain that subtree itself, forever, while continuing to receive everything else from upstream on a daily cadence.

Ownership therefore does not run around the repository boundary. It runs *through the tree*. Some directories in a service fork are upstream-owned and regenerated continuously. Other directories in the same repository, at the same commit, are fork-owned and must never be touched by synchronization. Part IV is about how that line is drawn and enforced, and it is the most consequential mechanism in the system.

What This System Is

Concretely, `osdu-spi` is a GitHub *template repository*. A service fork created from it inherits a complete set of workflows, composite actions, configuration schemas, rulesets, and build assets. The template owns the logic; each fork owns its data.

The system manages eight core OSDU services — partition, entitlements, legal, schema, file, storage, indexer, and search — with the explicit intent that adding a ninth adds no new patterns. Every fork runs the same workflows, and a fix made once in the template arrives everywhere as a reviewable pull request.

What it provides, in the project's own summary: one synchronization pull request per upstream state and never a pile of duplicates; conflicts that surface in an integration branch rather than on the production branch; releases correlated with the upstream version they contain; and initialization driven by a single issue reply.

What This System Is Not

It is not a deployment system. It does not provision Azure resources, does not run a Kubernetes cluster, and does not operate a live OSDU environment. It borrows one, briefly, to prove an image.

It is not a general-purpose fork tool. The filter engine that makes it work is specific to the shape of OSDU service repositories: a Maven multi-module tree with a provider directory convention, testing modules, and a core module.

It is not fully automatic. Several steps deliberately wait for a human. Merging a synchronization pull request is a human decision. Resolving a conflict is a human act. The automation's job is to make sure that when a human is needed, the system says so unambiguously and remembers where it left off.

And it is not finished. Part VIII describes a lane whose core is shipped and whose remaining gates are designed, tracked, and openly listed as unbuilt.

Part II: The Shape of the System

One Template, Many Forks

Every service fork needs the same workflows and the same actions, but a different upstream URL, a different service name, different Maven coordinates, and different repository settings. The template repository pattern separates those two things: the template owns the logic, and each fork owns its data in configuration files.

The propagation mechanism is a workflow called `Sync Template`, which runs in every fork daily at 08:00 UTC. It compares the fork's copy of the engineering system against the template's current state, and when the template has moved it opens a pull request in the fork. The fork owner chooses when to merge it.

This is the payoff the pattern exists for. Fixing a bug in `sync.yml` once is enough; the fix arrives in every repository as a reviewable pull request, rather than as a hand edit repeated eight times with a typo in the sixth. The propagation is asynchronous and review-based rather than immediate, which is the accepted cost.

Two Contexts in One Repository

The template repository contains two directories of workflows, and the distinction between them is load-bearing.

`.github/workflows/` is the **template's own** continuous integration. It runs only in `osdu-spi` itself: the template's development CI, its release management, its documentation build, its CodeQL analysis, its OpenSSF Scorecard run, and the two initialization workflows that configure a new fork.

`.github/template-workflows/` is **what forks receive**. At initialization these files are deployed into the fork as its `.github/workflows/`. This directory holds synchronization, cascade, cascade monitoring, validation, build, release, template sync, adoption, settings application, registry retention, integration cleanup, Dependabot validation, and the fork's own CodeQL configuration.

Before the split existed, every fork carried template-only infrastructure it never used. Separating the two is an instance of a rule the project states explicitly and applies repeatedly: *split what fails differently*. Template CI failing and fork initialization failing are unrelated events, and code that handles both in one file has two unrelated failure modes in one place.

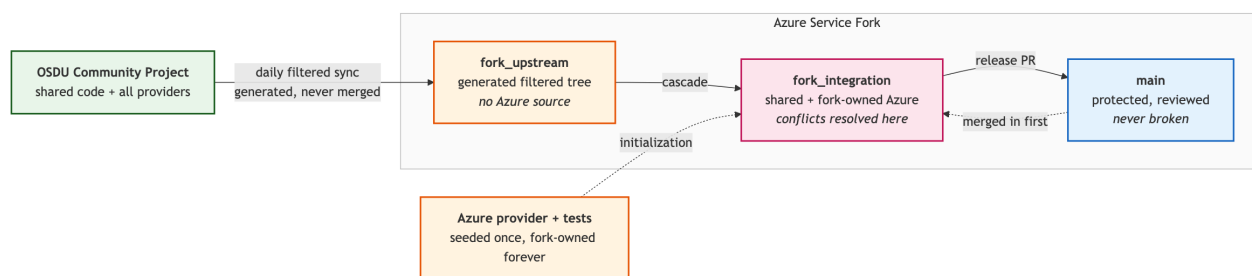
Three Branches, Three Different Jobs

A service fork maintains three long-lived branches. Their names are exact, and they are not interchangeable.

fork_upstream holds the upstream-derived tree. After the change described in Part IV it is not a mirror of upstream but a *generated* tree: shared upstream code with the non-Azure providers and upstream deployment machinery removed, and references to the fork-owned Azure modules injected. It is written only by synchronization and read only by the cascade. Nothing else touches it.

fork_integration is the workspace. It is where the generated upstream tree is combined with the fork-owned Azure source, where conflicts are resolved, where Maven versions are stamped, and where the build and tests run. It is allowed to break.

main is production. It is protected: pull requests only, one approving review, and two required status checks named exactly **CodeQL** and **Validation Summary**. It receives only changes that have already been integrated and validated somewhere else.



The three-branch flow. Upstream change enters a generated branch, is integrated and validated in a workspace branch, and reaches production only as a reviewed release pull request.

Why three rather than two, or one? The register considered all three options. A two-branch strategy is simpler. Merging upstream directly into the working branch is simpler still. Both were rejected for the same reason: they put conflict resolution and build failure on the branch the team is trying to ship from.

The project's own account of the payoff is worth quoting in substance. When integration breaks, the break is contained on **fork_integration** and fixed there, while feature work on **main** continues. Adding a stage did not slow anyone down. It removed the case where one upstream change stopped everybody at once.

The branches also differ in what is allowed to happen to them. **fork_upstream** accumulates: an unmerged synchronization pull request is not a problem, it is a queue. **fork_integration** is disposable and is reset to **main** after a release lands. **main** is never permitted to be broken.

Protection is applied from checked-in configuration. Two ruleset files, **default-branch.json** and **integration-branch.json**, are reconciled onto the repository weekly by a **Settings Apply**

workflow. The default-branch ruleset requires review and status checks on `main`. The integration-branch ruleset protects `fork_upstream` and `fork_integration` from deletion only — automation must remain able to force-push them.

Ownership Runs Through the Tree

The single most important structural fact about a service fork is that ownership is not uniform across the repository.

Upstream-owned, and regenerated on every synchronization:

- the service’s core module and shared code
- the community acceptance test module
- the shared build configuration those modules inherit

Fork-owned, and never written by synchronization:

- `provider/<svc>-azure` — the Azure provider implementation
- `testing/<svc>-test-azure` — the Azure integration test source
- `.github/` — the engineering system itself
- `build/` — the canonical Dockerfile and entrypoint
- `.spi/` — the acceptance descriptor described in Part VIII

A single commit on `main` therefore contains code with two different provenances and two different maintenance contracts. The synchronization mechanism must respect that line precisely, on every run, forever. Part IV is about how.

Two Tiers

The model extends one level further than the eight Microsoft-maintained service repositories.

A customer organization can run a true GitHub fork of a service repository through the same machinery, in a different mode: instead of filtering an upstream community project, their fork mirrors the service repository’s finished `main` verbatim. They build and release with their own credentials, against their own environment, and features they prove can return as contribution pull requests through the GitHub fork network.

The two tiers run the same workflows, the same three branches, and the same cascade. They differ only in what their upstream is and how synchronization treats what it receives. Part IX describes the second tier in full; it is mentioned here because several mechanisms in the intervening parts carry a “unless this is a mirror” clause, and that clause is otherwise inexplicable.

Part III: Becoming a Fork

Initialization Is a Conversation

A new service fork begins as a GitHub repository created from the template. The first push triggers a workflow named `Initialize Fork`, which detects that it is running in a fork rather than in the template, creates the system labels, and opens an issue titled **Repository Initialization Required**.

That issue is the entire user interface. A human replies to it with the upstream repository to track. A second workflow, `Initialize Complete`, triggers on the comment and does the actual work: it validates that the upstream repository exists and is reachable, records it in the `UPSTREAM_REPO_URL` variable, generates the filtered `fork_upstream` branch, creates `fork_integration`, seeds the fork-owned Azure trees, plants a filter configuration file, deploys the template workflows into the fork, applies repository settings and rulesets, and finally sets `INITIALIZATION_COMPLETE` to true.

The ordering of that last step matters. `INITIALIZATION_COMPLETE` is written last, so a run that fails halfway leaves the repository visibly uninitialized rather than half-armed. Several workflows check the flag before doing anything, which means a partially configured fork does not start synchronizing against an upstream it has not finished validating.

Re-running after a failure converges. Steps that already completed detect their own output and skip or reconcile, so recovery is a re-run rather than a cleanup.

Split What Fails Differently

Initialization is two workflows rather than one because the original single workflow failed in two unrelated ways.

Handling user input — parsing an issue comment, validating a URL, dealing with a person who pasted the wrong thing — is a different problem from setting up a repository, with different error conditions and different recovery. Combining them produced a file where a malformed comment and a GitHub API permission error arrived at the same error handler.

The split gives each half one responsibility and one failure mode. It is the same rule that separated the template's own workflows from the workflows forks receive, applied at a smaller scale.

The Bootstrap Problem

There is a circularity in a self-configuring template, and the project solves it with a directory.

Some logic must run in the fork's very first commit, before any synchronization could possibly deliver a fix to it. If that logic lives in the normal shared action directory and it has a bug, the bug

cannot be repaired through the ordinary propagation path, because the ordinary propagation path is the thing that is broken.

The system therefore maintains three distinct homes for executable logic, separated by when they must be available:

- `.github/actions/` holds composite actions that are synchronized to forks and updated through template sync. Most logic lives here.
- `.github/local-actions/` holds logic that must exist in a fork's first commit, before any sync could deliver a fix. It also holds the test harnesses.
- `.github/scripts/` holds durable helper scripts.

This is downstream of a more general rule about shell code. Bash embedded in workflow YAML cannot be run locally, duplicates across files, and is hard to debug — and the project had already discovered the same script existing in two places. Substantial shell now lives in a `.sh` file wrapped by a composite `action.yml`, so the same file runs from a workflow, from a developer's shell, and from a test harness. Each script carries a shebang, `set -euo pipefail`, and writes its results both to `$GITHUB_OUTPUT` and to standard output so it is debuggable outside Actions.

The extraction is deliberately uneven, and the register says so: initialization helpers and state management are extracted, while orchestration remains inline. One extracted action, `issue-state-manager`, currently has no workflow calling it.

The One-Time Seed

Initialization performs an operation that happens exactly once in the life of a repository and can never be repeated.

Because upstream intends to delete its Azure provider implementations, the fork must capture them first. Initialization seeds `provider/<svc>-azure` and `testing/<svc>-test-azure` from the newest upstream revision that still contains them. From that moment, those trees are fork property. Synchronization will never write to them again, and late upstream fixes to Azure source — if any arrive before the deletion — require a deliberate manual port.

Initialization is also the only place in the system where a textual merge across unrelated histories occurs. The generated upstream tree and the seeded Azure tree have no common ancestor, so joining them once requires an actual merge. Everywhere else, as Part IV explains, the system stopped merging on purpose. The register notes this exception precisely: the `-X theirs` merge survives only in initialization, for the first join of unrelated histories.

Adoption, the Other Door

A second entry path exists for the customer tier.

A customer organization does not instantiate the template. They create a real GitHub fork of a service repository — which, being a fork, already inherits every deployed workflow from its parent's `main` — and then run a workflow called `Adopt Fork` manually.

Adoption detects the fork's parent as its upstream, creates `fork_upstream` and `fork_integration` at the merge base, creates the labels, sets `UPSTREAM_REPO_URL`, sets `SYNC_MODE` to `mirror`, applies the rulesets, enables the workflows that apply to a mirror while leaving template sync and Scorecard disabled, and sets `INITIALIZATION_COMPLETE` last.

The reason customer repositories must be forks rather than template instances is a GitHub constraint with an architectural consequence, and it is explained in Part IX.

Settings as Code

Repository configuration is not clicked into a settings page. It is reconciled.

The rulesets live as JSON under `.github/rulesets/`, and a `Settings Apply` workflow re-applies them weekly, every Monday at 04:00 UTC. This means a well-meaning manual change to branch protection is reverted within a week, and it means a new required check can be rolled to every fork by editing a file in the template.

Initialization also configures the security posture: CodeQL analysis, Dependabot with a fork-appropriate configuration, and the rulesets. A fork is secured at creation rather than secured later by an audit.

Part IV: Generate the Branch, Do Not Merge Into It

This part describes the mechanism that makes the rest of the system possible. It is also the clearest example of the project abandoning an obvious approach after the obvious approach failed repeatedly in a specific, diagnosable way.

The Conflict That Kept Coming Back

The naive plan is straightforward and almost works.

Take upstream. Delete the provider directories the fork does not ship — `provider/partition-aws`, `provider/partition-gc`, the GitLab CI configuration, the upstream deployment machinery. Commit the deletions. Then, every day, merge the new upstream into your branch with a strategy option that prefers upstream's version of any conflicting file.

This fails, and it fails in a way that gets worse rather than better.

`git merge -X theirs` resolves *content* conflicts by preferring the other side. It does not resolve **modify/delete** conflicts at all. When upstream modifies `provider/partition-aws/pom.xml` and the fork has deleted it, Git does not know whether the answer is the modification or the deletion. It stops and asks.

Worse, the conflict is not a one-time cost that decays. It recurs on *every* synchronization, because as the merge base advances, each new upstream change to a deleted file produces the same unanswerable question again. And the natural human recovery — `git add -A` to accept the working tree and move on — silently restores the deleted files, quietly undoing the strip.

The project's summary of this is one sentence: *stripping the other cloud providers from a fork cannot be done by deleting them and merging upstream afterwards.*

The Reframe

The insight that resolves it is a change of category rather than a better merge strategy.

`fork_upstream` is written only by synchronization and read only by the cascade. No human commits to it. No feature branches off it. It has exactly one producer and one consumer.

A branch with those properties does not need to be *merged into*. It can be **generated**. If the content of `fork_upstream` is a pure function of the upstream tip and a filter configuration, then

synchronization can compute the correct tree from scratch every time and write it directly. Nothing is textually merged, so the modify/delete conflict class cannot occur — not “is handled,” cannot occur.

The project states the rule as a heading: *generate the branch; do not merge into it.*

Building a Tree with Plumbing

The implementation uses Git’s low-level commands rather than its porcelain, because the goal is to construct a specific tree object rather than to reconcile two working trees.

Synchronization reads the upstream default branch into a scratch index, checks that index out into a generation directory, runs the filter engine over the directory, and then writes the result as a commit with two parents:

```
GIT_INDEX_FILE=$SCRATCH git read-tree upstream/$DEFAULT_BRANCH
GIT_INDEX_FILE=$SCRATCH git checkout-index -a -f --prefix=$GEN/
upstream-filter --config .github/upstream-filter.yml --checkout $GEN
git commit-tree $TREE -p fork_upstream -p $UPSTREAM_SHA
```

The last line is the subtle one. The new commit’s tree is the filtered content, but its parents are the previous `fork_upstream` tip *and* the upstream commit it was derived from. The result is merge-shaped: `git log` shows a normal history, upstream commits are reachable, `git blame` works, and the relationship to upstream is recorded in the commit graph. But no merge algorithm ever ran. The shape is a record of provenance, not the output of a reconciliation.

What the Filter Removes

The filter’s configuration lives in each fork at `.github/upstream-filter.yml`, and the engine classifies every top-level entry, every testing module, every POM profile, and every FOSSA module declaration.

Removed from the generated tree:

- `provider/` — every provider except Azure
- `devops/` — upstream’s deployment machinery, including `devops/azure`, because the Stack owns its own charts
- non-Azure `testing/` modules
- `.gitlab-ci.yml` — upstream’s pipeline definition
- `<svc>-core-plus` — a module with no consumers after the strip

Kept, and continuously updated: the core module, the community acceptance test module, and the shared build configuration.

Injected: references to the fork-owned Azure modules, so the generated tree's build wiring points at code that lives only on `fork_integration` and `main`.

That last point deserves emphasis. The generated branch contains *references* to Azure modules but not the Azure modules themselves. `fork_upstream` is provider-less by construction. This is why validation builds it with a Maven profile of `core` alone, a detail Part VI returns to.

The rejected alternatives are recorded with their genuine advantages. Sparse checkout or a custom merge driver would be less invasive but leave unwanted paths in the tree. Stripping `testing/` wholesale would produce a smaller tree but lose the Azure integration tests. Keeping `devops/azure` would preserve upstream's pipelines, but the Stack does not use them. Emitting the Azure POMs into the generated tree would simplify version stamping, but would reintroduce exactly the modify/delete conflicts the design exists to eliminate.

Halt on the Unknown

The filter classifies everything. When it encounters something the configuration does not name — a new top-level directory, an unrecognized testing module, a POM profile nobody declared — it does not guess.

It exits with code 2, and the run stops. An issue is opened carrying the labels `sync-failed` and `human-required`, naming what it could not classify.

This is a deliberate and somewhat expensive choice. The cheap alternative is a default: keep unknown things, or drop them. Both are wrong in a way that is invisible. Defaulting to *drop* means a new shared upstream module is silently deleted from the fork and nobody notices until a build fails for an unrelated-looking reason. Defaulting to *keep* means upstream's new AWS provider quietly lands in an Azure fork.

The project generalizes the rule across two components — the filter and the acceptance resolver in Part VIII both refuse unknown inputs — and states the reasoning in one line: *guessing would have been convenient exactly once and wrong forever after*.

Provenance in Two Trailers

Every generated commit carries two Git trailers.

`Upstream-Sha` records the exact upstream commit the tree was derived from. This is what makes “which upstream state is this” answerable without inference.

`Filter-Rev` records the filter's identity: the engine version joined to the first twelve hex characters of a SHA-256 over the configuration file's bytes. Two commits with the same `Upstream-Sha` and the same `Filter-Rev` are guaranteed to have the same tree. Two commits with the same `Upstream-Sha` and *different* `Filter-Rev` values are the same upstream state seen through different rules.

That second case is the one the trailer exists for. Because the generated tree is a function of both the upstream tip and the configuration, a change to the configuration alone changes the correct output even when upstream has not moved. The duplicate-suppression logic in Part V depends on this: its cache key hashes the configuration file, the filter engine, and the generation script together, so editing the filter is recognized as a change rather than dismissed as a no-op.

Proving the Filter Before Trusting It

Generating a tree instead of merging one trades a loud failure for a quiet one. A merge conflict stops the world and demands attention. A filter bug produces a perfectly clean commit containing the wrong tree.

The register names this as the cost of the approach, and the response is verification built into the engine, which runs in four modes: `generate`, `verify`, `stamp`, and `seed`.

Verification is of three kinds. **Negative** checks assert that things which should have been removed are absent. **Positive** checks — configured as `expected_kept` — assert that things which must survive actually did, which is the check that catches an over-aggressive rule. And a **non-mutating re-run** compares the engine's output against the current tree to raise an alarm when a new upstream module appears, before it can be silently mishandled.

There is also a manual escape hatch, used during the partition service's cutover: if the fork-owned trees are ever damaged, they can be restored from the previous commit directly, because they are ordinary tracked files.

```
git checkout HEAD -- provider/partition-azure testing/partition-test-azure
```

What the Fork Now Owns Forever

The consequence of Part IV is stated plainly in the register, including the part that hurts.

The Azure trees stop receiving upstream changes. If upstream fixes a bug in its Azure provider before deleting it, that fix does not arrive; somebody must port it deliberately. The cascade takes on responsibility for version wiring, because the fork-owned POMs must be kept consistent with upstream-derived coordinates that now move independently of them. And a bug in the filter manifests as a wrong tree rather than as a conflict, which is why the verification modes exist.

What is bought is a synchronization that cannot produce the recurring conflict class, runs unattended daily, and yields a reviewable pull request rather than a merge session.

Part V: The Cadence

Parts III and IV described a fork's creation and the mechanism that keeps its upstream branch correct. This part describes what actually happens, day after day, once the machinery is running.

A Sync That Remembers

Every day at midnight UTC, the `Sync Upstream` workflow runs in each fork. It fetches the upstream tip, generates the filtered tree as described in Part IV, and — if anything changed — opens a pull request into `fork_upstream` from a branch named `sync/upstream-YYYYMMDD-HHMMSS`.

The pull request is titled  `Sync with upstream <version>` and carries the commit list in its body, capped at 200 entries and 60,000 characters. A companion tracking issue titled  `Upstream Sync Ready for Review` is opened with the labels `upstream-sync` and `human-required`.

The interesting engineering here is not the synchronization. It is the memory.

A scheduled workflow runs on a fresh, stateless runner. It has no recollection of the previous day. A synchronization that cannot remember what it already did opens a duplicate pull request every single day that a human takes to review the first one — which is how the system originally behaved, producing notification fatigue, branch clutter, and genuine confusion about which pull request was current.

The obvious places to keep state were each rejected for a concrete reason. Git notes bring their own merge conflicts. Git config does not survive between hosted runner invocations. External storage adds a dependency and a failure mode. Closing the old pull request and opening a new one destroys the continuity of any review already in progress. Leaving both open is the problem itself.

The chosen answer is that durable state lives in GitHub's own metadata, split across three places by lifetime:

- A **hidden marker** in the tracking issue body, `<!-- upstream-sha: ... -->`, records the upstream SHA that the currently open synchronization is for. It lives exactly as long as the open synchronization does.
- The **`upstream-sync` label** is how the workflow finds its own branches, pull requests, and issues without guessing at names.
- A **repository variable**, `SYNC_LAST_EVALUATED_SHA`, records the last upstream state that was evaluated and found to produce no change in the fork's tree. Its value is not a bare SHA but `<upstream sha>:<generation revision>`, so a change to the filter configuration invalidates it even when upstream has not moved.

From those three, the daily decision is a four-case table:

Existing sync PR	Upstream changed	Action
No	Yes	Create the branch, the pull request, and the tracking issue
Yes	No	Leave everything untouched
Yes	Yes	Force-push the existing branch; update the title and body
No	No	Do nothing

Abandoned branches — more than 24 hours old with no associated pull request — are cleaned up.

The error handling is worth noting because it follows the same philosophy as the filter. A GitHub API read failure **fails the workflow** rather than proceeding on an assumption; guessing that no pull request exists when the API merely timed out would create the duplicate the mechanism exists to prevent. A missing or malformed marker compares as *changed*, which is the safe direction, and is repaired on the next write. A failed variable write costs one repeated evaluation, not a failed synchronization.

The whole mechanism is tested outside GitHub Actions, by a shell harness that exercises malformed lookups, abandoned-branch cleanup, legacy and current and CRLF-corrupted markers, malformed SHAs, stale generation revisions, unreadable variables, and workflow ordering.

The Meta Commit

Release Please, the tool that computes versions and changelogs, requires Conventional Commits — messages shaped like `feat:` or `fix:` or `chore:`. Upstream OSDU does not write them, and never will.

The obvious fixes all involve rewriting history: squash the upstream range into one properly formatted commit, or rebase and reword. Both destroy the upstream history, and with it `git blame` and any ability to bisect a regression to an upstream change.

Instead, synchronization *adds* one synthetic commit summarizing the range, and derives its type by a fixed rule over the commits it contains:

breaking beats `feat`, `feat` beats `fix`, and anything unclassifiable counts as `fix`.

A rule rather than a judgment. The same input always produces the same version bump, the full upstream history remains intact underneath, and versioning stays automatic.

The project frames the general lesson as *add metadata instead of rewriting history*.

The Cascade

Merging the synchronization pull request into `fork_upstream` is a human decision. What happens afterward is the cascade, and it is where integration actually occurs.

The `Cascade Integration` workflow merges in a specific order. First it merges `main` into `fork_integration`, bringing the workspace up to date with whatever production already contains. Only then does it merge `fork_upstream` on top. Reversing this would integrate upstream against a stale baseline and produce conflicts that are artifacts of the ordering rather than real disagreements.

It then asserts that the fork-owned paths are intact — the Azure provider and test trees must be present and unmodified by the merge — stamps the fork-owned POM versions to match the upstream-derived coordinates, and runs the full build and test.

On success it opens a pull request into `main` from a temporary branch named `release/upstream-YYYYMMDD-HHMMSS`, labeled `validated`. On conflict it opens an issue titled  `Cascade Conflicts: Manual Resolution Required` and stops. On a build or test failure it opens  `Integration Validation Failed: Build/Test Errors` and stops.

In both failure cases the work is now sitting on `fork_integration`, isolated, where a human can fix it without touching `main` and without blocking anyone's feature work.

The review is deliberately **asymmetric**. Not every hop requires the same scrutiny. Merging a generated upstream tree into an integration workspace is a low-stakes, reversible act; promoting a validated result to production is not. Making every stage symmetric would be simpler to describe and slower and noisier to live with.

Labels Are the State Machine

A cascade spans hours or days and crosses many stateless workflow runs. It needs somewhere to keep its state.

The first attempt used issue assignment — assign the issue to a person, and the presence of an assignee means somebody owns it. This failed on organization accounts, on renamed users, and on ordinary API errors.

Labels have none of those problems and are queryable from any workflow without a lookup. So labels became the state machine, not merely decoration:

`cascade-active` means a cascade is in flight. `cascade-blocked` means it stopped and needs help. `cascade-ready`, `cascade-failed`, and `cascade-escalated` mark the remaining positions. `validated` marks a result that passed integration. `upstream-held` marks work queued behind an active cascade. `integration-blocked` marks a stalled integration. And `human-required` means exactly what it says.

That last label carries a second, less obvious function: **its removal is a signal**. When a human fixes a conflict and deletes the `human-required` label, that deletion is what tells the system it may try again.

The tracking issue, meanwhile, is not just a notification. It is the audit trail for a process spanning days, and it is where the hidden state marker lives.

Human-Triggered Steps Need a Sweeper

The cascade is meant to be started by a person after reviewing the synchronization pull request. People forget. Transient failures leave work stuck halfway.

`Cascade Monitor` runs every six hours. It compares `fork_upstream` against `fork_integration`, finds merged synchronization pull requests with no corresponding cascade and starts one, retrieves cascades whose `human-required` label has been removed, escalates conflicts that have been stale for 48 hours, and reports overall cascade health.

The project is careful about what this is: *a safety net behind the manual path, not a replacement for it*. The sweeper does not make the cascade automatic. It makes forgetting non-fatal.

Cleaning Up After Itself

When a release pull request merges into `main`, an `Integration Branch Cleanup` workflow resets `fork_integration` back to `main`.

It triggers on a merged pull request to `main` that either carries the `upstream-sync` label, came from `fork_integration`, or came from a branch beginning with `release/upstream-`. It also closes any `upstream-held` issues, releasing work that was queued behind the cascade that just finished.

This is what makes `fork_integration` genuinely disposable. It never accumulates history of its own, so it never becomes a third thing to maintain.

Releases, and the Tag That Says Which Upstream

`Release Management` runs on every push to `main`, except when the push touched only `.github/**`.

`Release Please` computes the next semantic version from the conventional commits since the last release — including the meta commit that synchronization contributed — and opens or updates a release pull request containing the version bump and the generated changelog. Merging that pull request publishes the release.

Then two things happen that are specific to a fork.

First, a **correlation tag** is created, formatted as `<release-tag>-upstream-<upstream-version>`. This is the answer to a question a fork must be able to answer and a normal repository never faces: *which upstream release does this fork release contain?* Without it, that question is archaeology.

Second, the container image is **retagged, not rebuilt**. Validation already built and pushed an image identified by its commit SHA. The release adds the semantic-version tag to that existing image. The bits that were tested are the bits that ship, because they are literally the same bits. The workflow polls for the source image — up to 60 attempts at 20-second intervals, roughly twenty minutes — rather than assuming it is already there. The acceptance test image is tagged alongside it, which is what makes “run the tests that shipped with release X” a durable operation months later.

The release workflow has no manual trigger. A release is a consequence of merging to `main`, not an action someone takes.

Part VI: Building Someone Else's Code

Java 17, Maven, and a Repository You Do Not Own

OSDU services are Java and Maven projects, so the build path is fixed: Java 17 from Temurin, Maven, and JaCoCo 0.8.11 for coverage. The logic is implemented once as composite actions — `java-build` and `java-build-status` — and every fork calls the same ones.

The build depends on a Maven repository the project does not control. OSDU's shared artifacts are published to the Open Group's community GitLab instance, and the fork's Maven settings point at it:

```
https://community.opengroup.org/api/v4/groups/17/-/packages/maven
```

Access is anonymous, which removes a credential but not a dependency: if that host is unreachable, every build in every fork fails. The register lists this consequence without mitigating it.

Two other consequences are accepted explicitly. Java is pinned at 17 inside the action, so a fork needing a different runtime needs a different workflow. And a fork that is not a Java and Maven project is out of scope — Gradle and Bazel were both considered and rejected on the grounds that upstream is Maven and conversion work buys nothing.

Fast feedback for developers is a separate workflow. `Build` and `Test` runs on pushes to ordinary branches only, explicitly excluding `main`, `fork_integration`, `fork_upstream`, `sync/**`, and `dependabot/**`, so that feature work gets a quick build and coverage report without colliding with the branches the automation owns. Its artifacts are retained for two days.

The Profile Trap

This is a small detail with an outsized failure mode, and it is a good illustration of why the engineering system centralizes build invocation instead of letting each fork improvise.

Maven profiles can be marked `activeByDefault`. OSDU service POMs use this for the core module. The Azure fork only needs the core and Azure profiles, so the apparently correct command is `-P azure`.

It is wrong. **Maven deactivates every `activeByDefault` profile the moment any explicit `-P` is passed.** Building with `-P azure` therefore silently drops `core`, and the build fails to resolve `<svc>-core` — a confusing error a long way from its cause.

The rule adopted is consequently phrased in terms of what is never emitted: the build always passes a non-empty `-P`, and the default is `core,azure` rather than `azure`. A repository variable, `MAVEN_PROFILE`, overrides it for forks whose layout is unusual, and two such forks exist:

`entitlements`, whose provider sits at `provider/entitlements-v2-azure`, and `indexer-queue`, which carries provider modules in its default `<modules>` block.

There is one context-sensitive exception. Because `fork_upstream` is provider-less by construction (Part IV), building it with `core,azure` would fail — there is no Azure source there to build. Validation therefore selects the profile from the branch and the tier:

```
maven_profile: ${ { (github.base_ref == 'fork_upstream' || github.ref_name ==  
'fork_upstream')  
    && vars.SYNC_MODE != 'mirror')  
    && 'core' || vars.MAVEN_PROFILE || 'core,azure' }}
```

Read plainly: on `fork_upstream` in filter mode, build `core` only; otherwise use the fork's override if it has one, and `core,azure` if it does not. The `SYNC_MODE` clause is there because at the mirror tier `fork_upstream` *does* carry provider code, for reasons Part IX explains.

The accepted cost is that CI produces no signal at all about non-Azure providers. That is intentional — building four providers to learn about one wastes runner time — but it means an upstream change that breaks AWS is invisible here.

The Dockerfile Nobody Maintained

The original design assumed each service repository carried a usable Dockerfile. That assumption was false, and the counter-example is preserved in the register because it is instructive.

`provider/partition-azure/Dockerfile`, inherited from upstream, specified a base image of `openjdk:8-jdk-alpine` for a Java 17 service, copied a JAR named `partition-aks-1.0.0.jar` that no longer existed, and assumed a module-relative build context that CI does not use. It was not subtly wrong. It had not worked in a long time, and nobody had noticed, because nothing built it.

The ruling is that the engineering system owns one canonical `build/Dockerfile` and CI uses that one regardless of what a fork happens to carry. The image is built on `mcr.microsoft.com/openjdk/jdk:17-azurelinux`, pinned by digest. The Dockerfile itself is deliberately minimal — an `ARG JAR_FILE` and a `COPY` — and the knowledge of *which* JAR is supplied by the build machinery, defaulting to:

```
provider/<SERVICE_NAME>-azure/target/*-spring-boot.jar
```

with `SERVICE_TARGET_JAR` available to disambiguate when a module produces more than one. Application Insights instrumentation is added by `ADD --checksum` into `/opt/agents/` and attached at runtime by `build/docker-entrypoint.sh` only when `APPLICATIONINSIGHTS_CONNECTION_STRING` is present, so the agent is inert by default.

The image is built from the JAR that Maven already produced, rather than compiled inside the Dockerfile. Building inside would be more self-contained, but it duplicates the Maven build, discards the dependency cache, loses the coverage report, and is slower. The consequences accepted are that forks cannot easily diverge their Dockerfile, that stale in-repository Dockerfiles still exist but are unused, and that ARM64 validation on the push path pays for QEMU emulation on an amd64 runner.

A Public Registry, and a Setting That Cannot Be Fixed

Service images go to GitHub Container Registry, publicly.

The reasoning is narrow and situational. These images are test artifacts destined for a shared AKS cluster run by the Stack. Pushing from GitHub Actions with the built-in `GITHUB_TOKEN` requires no extra credential, and public visibility means the cluster pulls anonymously with no `imagePullSecret` to provision and no chart wiring to maintain. Azure Container Registry remains a viable fallback and was rejected as unnecessary rather than unsuitable.

The failure mode is specific and worth knowing. If a package's visibility flips to private, every cluster pull fails with `ErrImagePull`. The `Settings Apply` workflow checks visibility and reports it — but **cannot fix it**, because GHCR exposes no API for changing package visibility. The system can tell you the thing is broken and cannot repair it, which is a rare and honest thing for a register to record.

Images accumulate, so a `GHCR Retention` workflow runs weekly, keeping the newest five snapshot tags per branch and deleting SHA-tagged images older than thirty days.

Coverage Without a Threshold

JaCoCo runs and publishes a report. No threshold is enforced.

The register states this as an accepted consequence rather than an oversight: coverage is informational. For a fork whose majority of code is written by somebody else and regenerated continuously, a coverage gate would measure upstream's habits more than the fork's, and would fail builds for reasons no local change caused.

Part VII: Trust

Automation that can write to repositories, publish container images, and authenticate to Azure is an attack surface. This part covers how the system decides which code is allowed to run with which credentials.

Why the Default Token Is Not Enough

GitHub Actions provides every run with a `GITHUB_TOKEN`. Under Microsoft's organization policy that token is read-only, and the permission to create pull requests is disabled outright. Personal access token lifetimes are also constrained by policy.

This is not an inconvenience to be worked around; it eliminates the two obvious approaches. A synchronization workflow that cannot open a pull request cannot do its job, and a personal access token belonging to an individual is both against policy and a single point of failure tied to one person's employment.

The system uses **GitHub App installation tokens**. An organization-owned App is installed on selected repositories, and workflows needing elevated permissions mint a token at the start of the run using a pinned action:

```
actions/create-github-app-token@bcd2ba49218906704ab6c1aa796996da409d3eb1 # v3.2.0
actions/create-github-app-token@67018539274d69449ef7c02e8e71183d1719ab42 # v2.1.4
```

Two versions are in use: the newer in `settings-apply.yml` and `init-complete.yml`, the older in several other template workflows. Actions are pinned by commit SHA rather than by tag throughout, because a tag can be moved and a SHA cannot.

The credentials are two secrets, `RELEASE_APP_ID` and `RELEASE_APP_PRIVATE_KEY`, which must exist wherever a token is minted. The App needs read and write on contents, pull requests, administration, workflows, and variables — administration because it applies rulesets, workflows because initialization deploys workflow files.

The costs are real and recorded. Tokens expire after one hour, so a long-running job must be structured around that. Organization admin approval is required to install the App, and changing its permissions requires re-approval. A helper script, `rotate-app-key.sh`, exists because private keys need rotating.

Know the Event Context Before Building a Workaround

Cascade triggering was unreliable for weeks, and the cause was not a bug in the cascade.

GitHub's `pull_request` event runs the workflow file **as it exists on the pull request's branch**. When the cascade is triggered by a pull request whose branch does not yet contain the current workflow definition, the workflow that runs is the wrong one, or none at all.

`pull_request_target` runs the workflow from the **target** branch instead. Switching the trigger fixed weeks of unreliability with one line.

The project draws the general lesson as a heading — *know the event context before building a workaround* — because the alternative path was to build increasingly elaborate machinery around a misunderstanding of the platform.

That same context has a sharp edge, and the system ran into it. A job that checks out the pull request's head under `pull_request_target` is running untrusted code in a privileged context, and CodeQL flagged it as a cache-poisoning vulnerability. The response was structural: the validate-only Docker job **never sets a checkout ref** and never takes that lane at all.

The Guard Clause Is the Policy

Two jobs in the system hold real credentials. `docker-push` has `packages: write` on the registry. `deploy-test`, described in Part VIII, holds an Azure federated identity.

The rule is that credential-bearing jobs run only in trusted contexts, and the trust boundary is drawn at *who can put code in the branch being built*.

Excluded, always: `pull_request_target` runs, pull requests from external forks, Dependabot pull requests, and ordinary `workflow_dispatch` invocations. Allowed: pushes to `main` and `fork_integration`, cascade pushes to `fork_integration`, and pull requests whose head repository is the same as their base repository. Pushes to `fork_upstream` may take the non-credentialed build path only.

In practice the boundary is an `if:` clause carrying those conditions, repeated verbatim on each sensitive job. That repetition is deliberate, and the project defends it in one line: **the clause is the policy; a reviewer noticing a missing guard is not**. Any future credential-bearing job copies it.

The honest cost is that external contributors get no image push and no deployment signal on their pull requests. A maintainer-approved override input exists in the schema but is explicitly *not yet wired*, and the register says so rather than implying the capability exists.

A Required Check That Always Runs

A required status check has a name, and GitHub matches it literally. This creates a class of problem that has nothing to do with security and everything to do with mechanics.

The ruleset on `main` required a check named `CodeQL`. The CodeQL workflow's jobs had different names, and it skipped analysis entirely on configuration-only pull requests — correctly, since there was no code to analyze. A skipped job does not report a check. The pull request waited forever for a check that would never arrive.

Compounding it, the ruleset also used a `code_scanning` rule with an alert threshold, which waits for SARIF results. For a documentation-only change, no SARIF is ever produced, so the pull request blocked permanently.

The fix has two halves. The workflow gained a summary job named exactly `CodeQL` that runs with `if: always()` and reports the aggregate outcome by inspecting the results of the jobs before it — so the check always exists, whether analysis ran or was correctly skipped. And the `code_scanning` rule was removed from the ruleset entirely.

The alternative of uploading an empty SARIF file to satisfy the rule was rejected as misleading, which is consistent with the system's general posture: report the true state rather than manufacture a passing signal.

What was genuinely lost is worth stating. Blocking a merge on an alert severity threshold is no longer possible; findings are triaged through the Security tab rather than enforced by the ruleset. The register records that as a cost, not as a non-issue.

Dependabot, Split by Ownership

Dependency scanning follows the same ownership line as everything else.

The **template** scans what the engineering system owns: `.github` for GitHub Actions versions, daily at 08:00, and `/build` for Docker base images, daily at 08:30.

A **fork** scans only its application code — Maven ecosystems across the root, the core module, the acceptance test module, the Azure provider, and the testing modules — daily at 09:00, with updates restricted to **patch level only**. Both configurations cap open pull requests at five and label them `dependencies`.

Patch-only is the interesting restriction. Minor and major upgrades are excluded not because they are unimportant but because they conflict with upstream OSDU's own dependency choices; a fork that unilaterally advances a shared library diverges from the community build. The accepted cost is that minor and major upgrades must be scheduled deliberately rather than arriving automatically, and the register acknowledges the technical debt that implies.

Forks do not scan `.github` at all, because those updates arrive through template sync. During initialization, open Dependabot pull requests outside the fork's Maven ecosystem are closed, because a freshly created fork inherits the template's scanning configuration until its own replaces it — and Dependabot caches ecosystem lists, so the inherited configuration can produce noise before the swap takes effect.

Part VIII: Proving It Against a Real Environment

Everything to this point produces a container image. Nothing so far establishes that the image works.

This part describes the lane that closes that gap, the contract system that makes it possible across many repositories, and — in the project’s own accounting — which parts of it are shipped and which are designed but not yet built.

The Question Every Test Asks First

An OSDU acceptance test suite is not self-contained. Before it can assert anything, it must know where the service is, how to authenticate to it, and which identifiers to use: a gateway URL, a data partition, a tenant, a bearer token, sometimes a legal tag.

The subgroup-core wiki names the root cause of a decade of accumulated pain in one sentence, which this project quotes:

“The tests are in the repository; the knowledge of how to run them is not.”

Upstream kept that knowledge in four cloud providers’ pipelines. Severing those pipelines — which is exactly what Part IV’s filter does — orphans the tests. They exist, they compile, and nobody can run them.

Five Constraints That Rule Out the Obvious Answer

The obvious answer is to store the environment details in the repository as variables. Each of the following makes it worse.

The environment is disposable. The Stack is designed to be torn down and rebuilt weekly. Any environment value copied into a repository is stale by construction.

The machinery is multiplied. The same workflows synchronize from the template to eight service forks and onward to customer mirror forks pointing at *their own* environments with *their own* credentials. Nothing environment-specific or Microsoft-specific can be baked in.

Developers come first. The same tests must run against a personal Stack instance during development, with no CI in the path.

Releases must stay provable. “Run the tests that shipped with release X against environment Y” must remain a one-command operation months later.

CI must stay fast and honest. It must fail in seconds with a typed reason when the environment is not ready, and must never report green for a suite that did not run.

A working prototype had already demonstrated the failure mode. It pushed roughly sixteen repository variables into each fork, of which six went stale on every weekly rebuild, with no reconciliation loop — and it carried a sticky `DEPLOY_VALIDATED` boolean that never expired, meaning it kept vouching for an environment that no longer existed.

The project distills the requirement to a heading: **no environment values in the repository.**

Three Contracts, Three Owners

The answer separates what is known from who knows it.

Contract	Owner	Content
Facts — <code>spi info --json</code> , <code>spi status --json</code>	<code>osdu-spi-stack</code>	What the environment knows about itself: endpoints, tenant, data-plane client ID, partitions, Key Vault name, deployability, maintenance state, pinned services, seeded loads. Read per run, cached nowhere.
Descriptor — <code>.spi/service.yaml</code>	each service fork	What this service's suites need, symbolically. Reviewed in the same pull request as the test change.
Machinery — actions and <code>validate.yml</code>	<code>osdu-spi</code> (template)	The resolver joining descriptor against facts, the acceptance image build, the deploy lane, the required-check summaries. Synchronized to every fork.

The facts envelope is versioned — `apiVersion: spi.osdu.dev/v1` — so the contract can evolve without silent breakage.

The descriptor is fork-owned for a specific reason. The community acceptance test module is upstream-owned and regenerated on every synchronization, so nothing fork-specific can live inside it. The contract lives *beside* it, in `.spi/`, which the filter excludes from synchronization and CODEOWNERS protects.

The Descriptor

A descriptor declares named suites, each binding environment variables to *symbols* rather than values:

```

schemaVersion: 3
service: { name: partition, archetype: java-maven-azure }
tests:
  acceptance:
    type: maven
    path: partition-acceptance-test      # upstream module, kept by the filter
    mavenArguments: [test]
    bindings:
      HOST: { source: gateway }
      DATA_PARTITION_ID: { source: partition }
      PRIVILEGED_USER_TOKEN: { source: token }
    timeoutMinutes: 15

```

The source vocabulary is **closed**: `gateway`, `partition`, `openid`, `tenant`, `legalTag`, `keyvault:<name>`, `static`, `template`, `user`, `token`, `memberToken`, and `noAccessToken`. An unknown source kind is a hard failure naming the key — the same halt-on-the-unknown rule the filter follows in Part IV, applied by a different component for the same reason.

Certain environment names are refused outright, because a descriptor that could set them could hijack the runner: exact names like `AZURE_CLIENT_ID`, and anything prefixed `ACTIONS_`, `GITHUB_`, `RESOLVER_`, `RUNNER_`, or `SPI_STACK_`.

Resolution precedence is explicit environment, then facts, then a declared default, with `template` rendering last. That ordering is what lets an identical descriptor run in CI, against a developer’s personal environment, or pointed at a service running on a laptop.

The resolver’s exit codes are typed, and the distinction matters more than it appears: **2** is a descriptor violation, **3** is environment-not-ready, and **4** is an infrastructure contradiction. A test failure is none of these. Conflating “the environment was down” with “the code is broken” is precisely the ambiguity the taxonomy exists to remove.

What the Repository Is Allowed to Know

The entire per-repository configuration surface is five values.

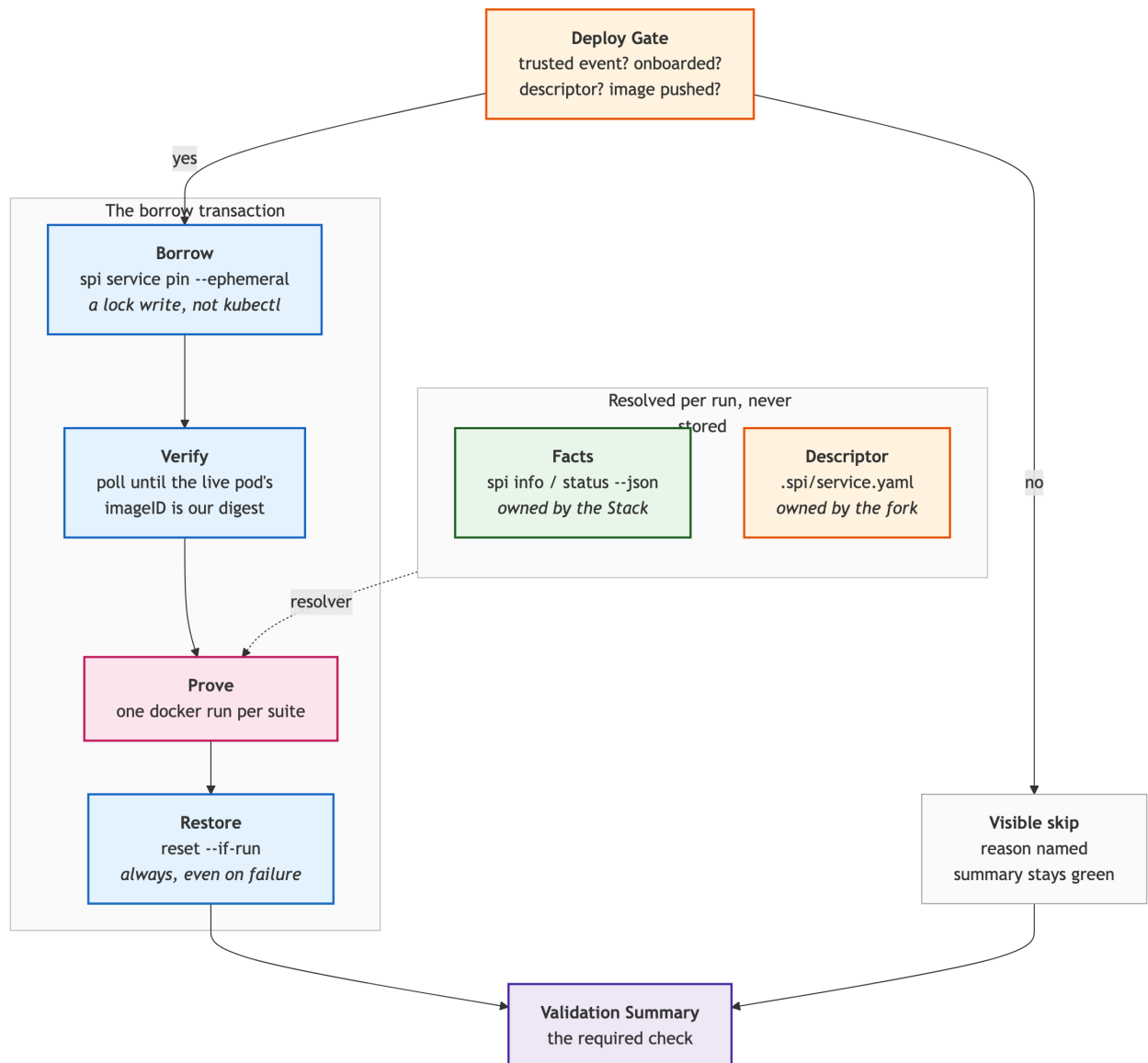
Value	Kind	Why it may live in the repository
<code>AZURE_CLIENT_ID</code> , <code>AZURE_TENANT_ID</code> , <code>AZURE_SUBSCRIPTION_ID</code>	secrets	The organization’s OIDC identity for its own environment. Stable across rebuilds.
<code>SPI_STACK_RESOURCE_GROUP</code> , <code>SPI_STACK_CLUSTER</code>	variables	The pointer. Stable across rebuilds by design.

Everything else — gateway URL, Key Vault name, storage account, client IDs, cluster details — is read from the facts contract per run. A weekly rebuild therefore costs **zero reconciliation** across every

fork and every mirror. That is the whole argument for discovery over push, and it is why the pointer is allowed to persist while the facts are not.

Borrow, Prove, Restore

The lane is one credentialed job, `deploy-test`, chained behind a gate job, `deploy-gate`, in the validation workflow. It runs on the fork's own pull requests and again on the push after merge, because a merge is a new build with a new digest.



The borrow transaction. The lane pins its own image into a running environment, proves it with the declared suites, and restores the canonical image whether or not the tests passed.

```

gate    → trusted event? onboarded? descriptor present? image pushed?
install → the spi release the environment reports (environment.stackVersion)
gate    → spi status --json : is the environment deployable?
borrow  → spi service pin --image ghcr...@sha256:... --ephemeral --run-id $GITHUB_RUN_ID
verify  → spi service verify : poll until the live pod's imageID is our digest
mint    → three tokens: RESOLVER_TOKEN, RESOLVER_MEMBER_TOKEN, RESOLVER_NO_ACCESS_TOKEN
bind    → resolver: descriptor × facts × tokens → <suite>.env
prove   → docker run --env-file <suite>.env <svc>--acceptance@<digest> (per suite)
restore → spi service reset --if-run $GITHUB_RUN_ID (if: always)
verdict → Validation Summary

```

Several steps carry more design than their single line suggests.

Deploy is a lock write, not a cluster mutation. The naive approach — `kubectl set image` — does not work in a GitOps environment, because Flux reverts it. The workaround of suspending Flux for the duration of CI was rejected by the Stack’s own decision register: it costs self-healing for as long as it lasts and it fails the first deploy after every rebuild. Instead, `spi service pin` performs a compare-and-set update to an `osdu-image-lock` ConfigMap, and Flux re-renders the release from it. GitOps stays alive throughout. In the Stack’s phrasing, *a lock write is the whole deploy*.

The pin returns before the image is live, because the fork’s identity deliberately holds no Flux write permission; the lock object’s watch label triggers reconciliation asynchronously. So the lane polls `spi service verify` until the running pod’s `imageID` matches the digest it pushed. A typed `lock_mismatch` means somebody else’s pin replaced this one mid-borrow.

Restore is ownership-checked. `reset --if-run` restores the recorded canonical image only while the live pin still belongs to *this* run, so a crashed job cannot “restore” over a newer sibling’s borrow. The step runs on `if: always()`, including after test failures. The project is candid that this is best-effort: a runner that dies outright can still strand a pin.

Not-onboarded is a visible skip, not a hidden one. When the pointer variables are absent, the gate reports “repository is not onboarded to a stack,” the lane does not run, and the summary check stays green. This lets one ruleset serve every fork, onboarded or not, without per-repository check configuration. The alternative — stripping the check from non-onboarded forks — was rejected because it hides an absence rather than explaining it.

The gate replaces the sticky boolean. Live status is consulted every run. The lane polls deployability every 20 seconds for up to ten minutes, and if the environment stays blocked it reports the reason, including maintenance.

Verdicts carry attribution. Environment-not-ready, infrastructure error, and test failure are distinct outcomes. A suite that collects zero tests is never a pass. An interrupted run is never a pass.

Concurrency is per service — group `spi-stack-<service>` with `cancel-in-progress: false` — so one service’s runs queue rather than cancel each other. The accepted cost is that a busy fork can

queue, and a different service borrowing the same environment can produce up to ten minutes of deployability polling. Each hold is capped by the suite's declared `timeoutMinutes`.

`fork_upstream` never enters the lane at all, because filter-mode builds produce no Azure image to deploy.

Secretless Callers

No long-lived credential exists anywhere in this path.

GitHub's OIDC token is exchanged for an Azure federated identity. The subject GitHub signs is exact — `repo:<org>/<fork>:environment:spi-stack` — which is why the GitHub environment exists at all. It is used for *identity protection*, not configuration storage, and it admits every branch: write access to the fork is the real boundary, and a pull request from another repository never receives an OIDC token in the first place.

The Azure roles are minimal: AKS Cluster User and Key Vault Secrets User. The Kubernetes permissions are narrower still — patch on the lock object restricted by `resourceNames` to `osdu-image-lock`, read-only on deployments, pods, and logs, and no create, no delete, and no secrets verbs at all.

Three bearer tokens are minted per run and reach the suites under fixed names: `RESOLVER_TOKEN` for the privileged caller, `RESOLVER_MEMBER_TOKEN` for a plain user with no admin rights, and `RESOLVER_NO_ACCESS_TOKEN` for a caller with no entitlements at all. The latter two exist so that negative authorization tests — *this user should be refused* — can actually be written. A developer mints the identical tokens with `spi token`, `spi token --member`, and `spi token --no-access`, which is what makes the developer loop genuinely identical to CI rather than merely similar.

Tests as Images

The acceptance suite is built as its own container image, `<svc>--acceptance:<sha>`, from the same commit as the service image, with Maven dependencies pre-warmed at build time.

The rationale is the “provable release” constraint: running the tests that shipped with a given release, months later, must stay one command. Bare Maven on a runner cannot offer that, because it re-resolves dependencies from a moving repository every time.

The project is precise about the limit of this guarantee. The image pins the *suite source* and a warmed local repository; it does not pin dependency resolution. The run is online, and version ranges in the upstream dependency graph can still resolve differently later.

One deliberate judgment call: the suite image runs **on the GitHub runner**, executing `docker run --env-file` against the public gateway, rather than as a Job inside the cluster. In-cluster would be closer to production networking, but it requires create and delete verbs that the fork identity

deliberately does not have. The accepted cost is that runner-to-gateway egress must remain open — true today, and flagged for revisit if the environment ever moves behind a private endpoint.

A detail from the developer documentation is a small security point worth preserving: the generated `.env` file is `docker run data, never shell code`. Maven does not read it, and sourcing it in a shell would evaluate secret values as shell commands.

Answers That Must Be Created

Some answers cannot be discovered, because nothing creates them. A brand-new environment has no legal tag and no entitlements for a test caller, and a suite that needs either simply fails.

The design separates seeded state into three tiers by lifetime and ownership:

- **Tier 1, Bootstrap** — the baseline the platform is unusable without: legal tag, partition configuration, entitlements groups, schemas, and the test-identity step. Reconciled by GitOps on every rebuild, idempotent by HTTP status (201 created, 409 already exists), with results published back as facts.
- **Tier 2, Named loads** — persistent shared data such as reference datasets. Labeled Kubernetes Jobs driven by a YAML registry, batch-loaded with duplicate skipping, run as the last stage of the rebuild pipeline rather than by hand.
- **Tier 3, Fixtures** — per-run, identity-shaped state, created at test start and torn down in a `finally` block, with orphans reaped by the next run.

The boundary rule is adopted verbatim from a predecessor project: *persistent state becomes a declared load; per-run identity becomes an ephemeral fixture*.

“Ready” and “seeded” are deliberately separate signals. A failed eighty-thousand-record data load should never block platform readiness.

What Is Not Yet Wired

The project’s own status line is that the core lane is built and specific items remain. They are listed here as the documentation lists them, with their tracking issues.

- **Key Vault materialization is not wired.** Descriptors may declare `keyvault:` bindings; the lane does not yet pass a secrets file. (#175)
- **Declared requirements are not enforced before borrowing.** A descriptor’s `requires.loads`, `requires.groups`, and `dependencies` are validated as syntax but not compared against published environment state, so a suite can still borrow an environment lacking the data it declared. The intended behavior is a typed refusal — *environment not seeded: reference-data* — instead of the industry-standard failure mode of an acceptance test returning zero results and a human reading a runbook. (#176)

- **A second image verification immediately before each suite is not implemented.** Verification runs after the pin, not again at suite start.
- **The drift tripwire is pending.** The design calls for a build-stage static check, needing no cluster and no secrets, that fails the build when suite code reads an environment variable the descriptor does not declare — printing the file, the line, and a paste-ready declaration. The point is that a new requirement surfaces at review time rather than as a red deployment three repositories downstream. (#156)

The documentation adds an unusual instruction about its own authority: where this design document and the two decision records disagree, **the records govern**. The design describes intended capabilities and is explicitly not a checklist of implemented gates.

Part IX: The Second Tier

Why It Must Be a True Fork

A customer organization running OSDU on Azure may need changes: a fix they need before the next release, an integration specific to their business, a feature they would like contributed back.

The mechanism for contributing back imposes the architecture. **A pull request's head branch must live in the target repository's fork network.** A repository created from a template is not in the service repository's fork network, no matter how identical its contents, and therefore can never open a pull request against it.

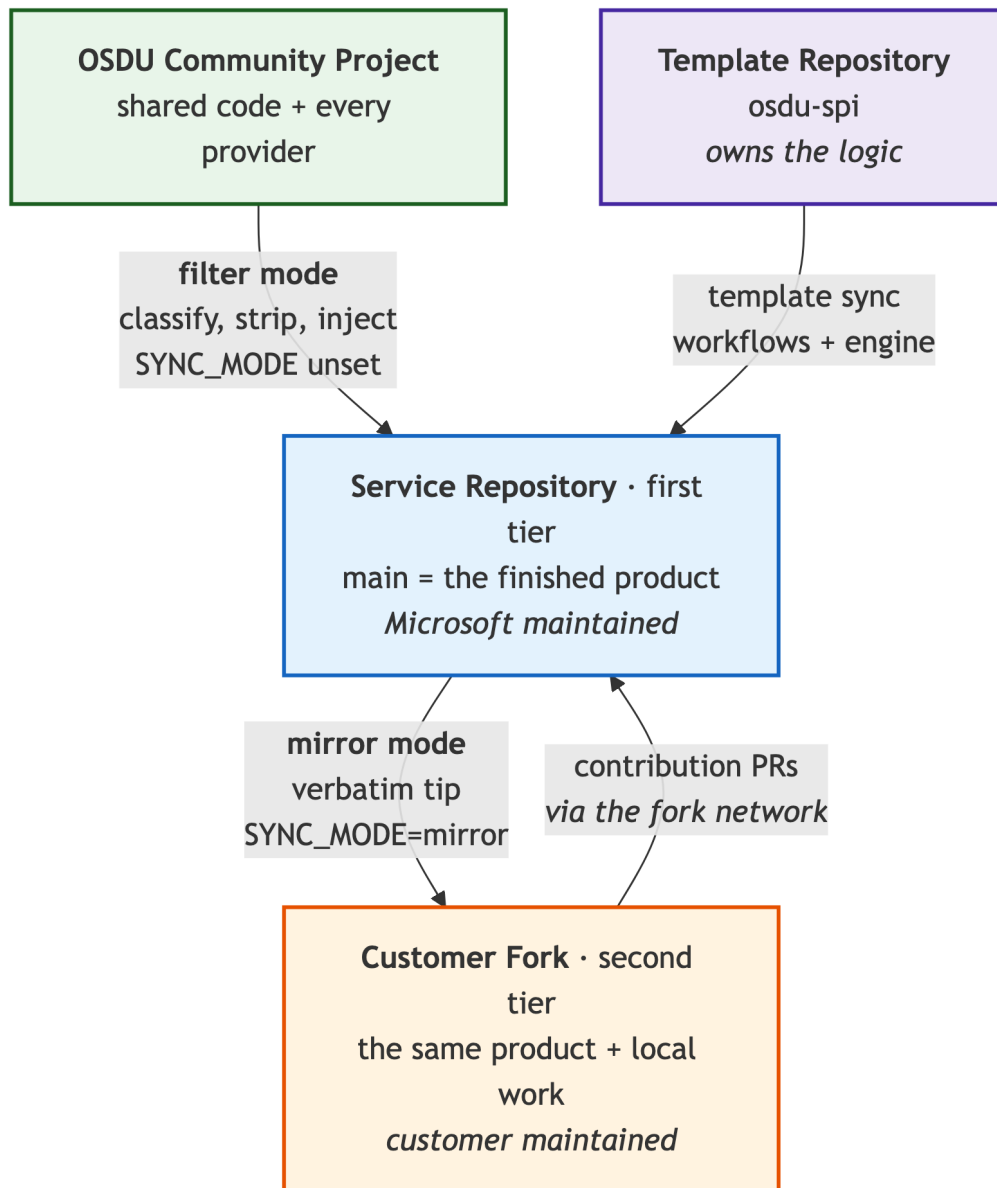
That single GitHub constraint settles the creation mechanism: customer repositories are real forks, never template instantiations. A convenient side effect is that a fork inherits every deployed workflow from its parent's `main` at the moment of creation, which is what makes adoption lightweight — the machinery is already there before the adoption workflow runs.

Mirror Mode

The upstream filter reduces a sprawling community project to the Azure product. Pointed at a service repository's `main`, it would be actively destructive.

That branch **is already the product**. Running the filter against it would halt on `.github/`, on `build/`, and on the changelog, none of which its configuration classifies — and it would strip `provider/`, which is the code a downstream consumer needs most.

So the second tier does not filter. `SYNC_MODE=mirror` takes the upstream tip **verbatim**, through the same commit plumbing described in Part IV: the same merge-shaped commits, the same `Upstream-Sha` trailer, and a sentinel `Filter-Rev: mirror`. Provenance and no-op semantics are identical at both tiers while the filter engine never runs at all.



The two tiers. A community project is filtered into a service repository; the service repository is mirrored verbatim into a customer fork, which can contribute back through the fork network.

The tier must be selected by a variable because **it cannot be inferred from the tree**. The filter configuration file exists at both tiers — at the customer tier it arrives *through the mirror itself* — so its presence says nothing, and deleting it would create a permanent difference against upstream that every subsequent sync would try to reconcile. One repository variable, set once at adoption, is the whole selector. Unset means existing behavior, so introducing the second tier changed no existing repository.

What Changes, and What Does Not

Both tiers run the same workflows, the same three branches, the same cascade, and the same release flow.

	First tier (service repository)	Customer tier (mirror fork)
Created by	Template instantiation + initialization	GitHub fork + Adopt Fork
Upstream	The upstream community project	The service repository's <code>main</code>
Sync behavior	Filter: classify, strip, inject	Mirror: verbatim tip
<code>SYNC_MODE</code>	Unset (or <code>filter</code>)	<code>mirror</code>
Fork-owned content	Azure provider, Azure tests, <code>.github/</code> , <code>build/</code>	Local feature work only
Cascade path assertion and POM stamping	Active	Off — no ownership split to defend
Template sync	Active	Off — updates arrive through the mirror
Sync PR build profile	<code>core</code> only	Full profile set

Two of those rows are consequences of the same fact. At the first tier `fork_upstream` is provider-less, so it builds `core` alone and the cascade must defend an ownership boundary. At the mirror tier `fork_upstream` carries the complete product including `provider/`, so it builds everything and there is no boundary to defend.

Template synchronization is off at the second tier because it would be redundant and conflicting: workflow updates already arrive as part of the mirrored product.

The Round Trip

The full development loop across both tiers:

1. **Upstream change.** The community project merges something. The first tier's filtered sync brings it in; the cascade integrates and releases it to the service repository's `main`.
2. **Mirror in.** The customer fork's next sync mirrors the new `main` verbatim; their cascade merges it with their local work.
3. **Local feature.** The customer develops and proves a feature in their own fork, with their own build, against their own environment.
4. **Contribute back.** The customer opens a pull request from their fork to the service repository. A maintainer approves the run and sees real build signal — while credential-bearing jobs stay off the external head, exactly as Part VII requires.

5. **Round trip.** The merged feature reaches the service repository's `main` and returns to the customer through their next mirror sync, where the cascade converges their local copy with the incoming change.

What the Second Tier Costs

Three consequences are recorded rather than hidden.

Mirror-tier repositories carry the same release automation, so **release conflicts happen**. `CHANGELOG.md` and `.release-please-manifest.json` are edited on both sides of the mirror, and those conflicts are shallow but real and sometimes need manual resolution. Disabling release automation at the mirror tier was considered and rejected, because it would remove the customer's release autonomy — which is much of the point of having their own fork.

Upstream deletions mirror through silently, by design. Mirror mode is verbatim; it does not second-guess a removal.

And mirror-tier validation **needs build credentials earlier** in the onboarding process than the first tier does, because their `fork_upstream` carries provider code and therefore builds the full profile set from the start.

Part X: The Things That Are Not True

A collection of statements that sound correct, are frequently assumed, and are wrong. Each is a real misconception this system's documentation takes the trouble to correct.

"fork_upstream is a mirror of upstream." Not at the first tier. It is a *generated* tree: upstream's shared code with non-Azure providers and deployment machinery removed and Azure module references injected. It has never contained the fork's Azure source, and it is not a faithful copy of anything. Only at the customer mirror tier is it verbatim, and that is what the `SYNC_MODE` variable selects.

"The daily sync merges upstream into the fork." Nothing is merged. The tree is computed from the upstream tip and written directly with Git plumbing. The resulting commit is *merge-shaped* — two parents, upstream reachable — but that shape is a record of provenance, not the output of a merge algorithm. This is the entire reason the recurring modify/delete conflict cannot occur.

"The filter removes the other providers from the repository's history." It removes them from the generated tree. Everything that was ever committed remains in history and remains reachable. The filter controls what the branch contains, not what the repository remembers.

"A green check on the sync pull request means the fork builds." It means the *provider-less* tree builds, with a Maven profile of `core` alone. There is no Azure source on `fork_upstream` to compile. Whether the fork actually builds is established later, in the cascade, on `fork_integration`.

"-P azure builds the core and Azure profiles." It builds Azure only. Maven deactivates every `activeByDefault` profile the moment any explicit `-P` is passed, so `core` silently drops out and module resolution fails somewhere that looks unrelated. The profile is always written as `core,azure`.

"The cascade runs automatically after a sync." It is designed to be triggered by a person, after they review the sync pull request. `Cascade Monitor` sweeps every six hours to catch cascades that were forgotten or stuck, but the documentation is explicit that the sweeper is a safety net behind the manual path and not a replacement for it.

"Removing a label is housekeeping." Removing `human-required` is a control signal. It is what tells the monitor that a human has finished fixing the problem and the cascade may be retried. Labels here are the state machine, not decoration.

"Upstream bug fixes to the Azure provider will eventually arrive." They will not. The Azure provider and test trees were seeded once and are fork-owned from that moment. Synchronization never writes to them again. A late upstream fix to Azure source requires somebody to port it deliberately.

“A release rebuilds the container image.” It retags one. Validation already built and pushed an image identified by its commit SHA; the release adds a semantic-version tag to that same image. The bits that were tested are the bits that ship, because they are the same bits.

“A passing CodeQL check means CodeQL analyzed the code.” Not necessarily. CodeQL is the name of a summary job that always runs and reports the aggregate outcome. On a documentation-only or configuration-only pull request, analysis is correctly skipped and the summary still reports success. This is deliberate — it exists precisely so those pull requests are not blocked forever — but the check’s name promises more than its presence delivers.

“A green Validation Summary means the change was deployed and tested against a live environment.” Only if the fork is onboarded to one. If the pointer variables are absent, the deploy gate reports that the repository is not onboarded, the lane does not run, and the summary stays green. The visible skip is intentional, so that one ruleset can serve every fork; but green does not by itself mean proven.

“A descriptor’s declared requirements gate the lane.” Not yet. `requires.loads`, `requires.groups`, and `dependencies` are validated as syntax but are not compared against the environment’s published state before borrowing. A suite can still run against an environment missing the data it declared. This is tracked, and the documentation says plainly that the design is not a checklist of implemented gates.

“Coverage is enforced.” JaCoCo runs and reports. No threshold gates anything.

“External contributors get the full pipeline.” They get the build. They do not get an image push and they do not get a deployment signal, because credential-bearing jobs are excluded from external heads by design. A maintainer-authorized override is described in the schema and explicitly not wired.

“If the tooling can detect a problem, it can fix it.” `Settings Apply` checks whether the container registry package is public and reports when it is not. It cannot correct it, because no API exists to change package visibility. Detection and remediation are separate capabilities, and this system has one without the other in at least one place.

“The AI-generated pull request descriptions were AI-generated.” They were not. For the entire life of the reference fork, the tool crashed on every invocation and the deterministic fallback template produced every description that was ever published — while the workflow reported success and reviewers merged without noticing. That story is the subject of the next part.

Part XI: Writing Down What Was Learned

The project keeps two documents about its own decisions, and the relationship between them is unusual enough to be worth describing.

A Register Organized by Question

The first is a register of thirty-eight architectural decision records. It is organized by *the question each record answers* rather than by chronology, and each record carries a stated impact level.

The numbering runs from 001 to 041 with three numbers — 017, 032, and 034 — retired and deliberately left unused. That detail is a small piece of intellectual honesty: rather than renumbering to stay contiguous and thereby invalidating every existing citation, the corpus leaves the gaps and says why they are there.

The records cover fork management, synchronization and cascade, release automation, template propagation, workflow patterns, and most recently continuous integration and deployment. Each preserves not merely what was chosen but what was rejected, with the rejected option's genuine advantage stated in a line. That is why this document can describe trade-offs as the project's own reasoning: the reasoning was written down at the time.

A Second Document, Derived Rather Than Authoritative

The second is a file of learnings — short essays, one lesson each, written in the first person plural, each ending in a pointer to the decision record that holds the detail.

The separation is the interesting part. The register states what is true. The learnings state what was *understood*, and they are explicitly derived rather than authoritative: when the two could disagree, the record governs. This frees the learnings to be narrative, to describe a failure in the order it actually unfolded, and to generalize — things a decision record is a poor container for.

The headings alone carry most of the argument, and several have become the operating principles of the system:

Isolation is what lets the team keep shipping. Split what fails differently. Labels are the state machine. State that must survive a runner lives in GitHub metadata. Add metadata instead of rewriting history. Know the event context before building a workaround. Enforce trust boundaries in the workflow, not in review. Generate the branch; do not merge into it. Halt on the unknown. No environment values in the repository.

The Most Useful Entry Is About a Failure

One learning is worth reproducing at length, because it is the clearest example of the corpus's value and of the habit that produces it.

The system once generated pull request descriptions with an AI model, designed to degrade gracefully to a structured template when the model was unavailable. The design was reasonable. The graceful degradation worked exactly as specified.

The outcome, recorded plainly: the tool **crashed on every single invocation for the entire life of the reference fork**. The fallback was not a fallback; it was the only path that ever ran. The workflow reported success throughout. Reviewers read the template-generated descriptions and merged them without complaint. Nobody noticed for months.

The lesson the project draws inverts the original intent:

A fallback that cannot be distinguished from success is not resilience, it is a blindfold.

The response was not to fix the integration. Descriptions became deterministic, and the commit classification that drives versioning became a fixed rule rather than a judgment. The register also records why the feature was not simply replaced: the model platform it used was retired, the coding-agent API rejects the App installation tokens this system authenticates with, and the remaining option has no API surface at all. Reopening the question requires a provider clearing all three constraints, and none currently does.

Three things make that entry unusually good. It names a date and a pull request. It states an outcome that reflects poorly on an earlier decision by the same authors. And it generalizes to a rule that applies well beyond the feature that produced it.

The Habit Behind It

Read together, the two documents show a consistent posture that explains most of this system's design.

The recurring instinct is to **refuse to proceed on an assumption**. The filter halts on an unclassified path rather than defaulting. The resolver halts on an unknown source kind. The synchronization state manager fails the workflow on an API read error rather than concluding that no pull request exists. The deploy gate consults live status every run rather than trusting a stored flag — a flag whose predecessor kept vouching for an environment that had been deleted.

The second instinct is to **put the enforcement where it cannot drift**. The trust boundary is a guard clause in the workflow, not a convention reviewers are expected to notice. Branch protection is a checked-in ruleset reconciled weekly, not a settings page someone configured once. The Dockerfile is

owned centrally, because the distributed version had been broken for a long time without anyone knowing.

The third is to **make state durable and observable**. A process spanning days across stateless runners keeps its state in labels, issue markers, and repository variables, in the open, where a human can read it and a workflow can query it.

It is worth considering how much of this document's usefulness comes from the fact that someone wrote those lessons down while they still stung.

Glossary

Adopt Fork The workflow that onboards a customer-tier repository: it detects the parent as upstream, creates the branch topology, and sets `SYNC_MODE=mirror`.

Borrow, prove, restore The deployment lane's transaction: pin a freshly built image into a running environment, run the declared acceptance suites against it, and restore the canonical image unconditionally.

Cascade The integration phase. After a synchronization pull request merges into `fork_upstream`, the cascade merges `main` into `fork_integration`, merges `fork_upstream` on top, validates, and offers the result to `main` as a release pull request.

Cascade monitor A workflow running every six hours that detects missed, stuck, or repaired cascades and resumes them. A safety net behind the manual path, not a replacement for it.

Correlation tag A Git tag formatted `<release-tag>-upstream-<upstream-version>`, recording which upstream release a fork release contains.

Descriptor `.spi/service.yaml`. The fork-owned declaration of what a service's test suites need, expressed as symbolic bindings rather than values. Excluded from synchronization and protected by CODEOWNERS.

Deploy identity A managed identity, one per environment, federated to a specific fork and GitHub environment. Holds least-privilege Azure roles and namespace-scoped Kubernetes permissions.

Facts The environment's machine-readable self-description, published by the Stack's CLI as `spi info --json` and `spi status --json` under `apiVersion: spi.osdu.dev/v1`. Read per run, never cached.

Filter-Rev A commit trailer identifying the filter that produced a generated tree: the engine version joined to a hash of the configuration file. The sentinel value `mirror` marks the customer tier.

fork_integration The integration workspace branch. Where upstream and fork-owned code are combined, conflicts resolved, versions stamped, and validation run. Allowed to break; reset to `main` after each release.

fork_upstream The upstream-derived branch. At the first tier it is a generated, filtered tree with no Azure source. At the mirror tier it is a verbatim copy of the parent's `main`. Written only by synchronization, read only by the cascade.

Generated branch A branch whose content is computed from inputs rather than produced by merging. The central mechanism of this system.

Halt The filter's and the resolver's response to an input they cannot classify: stop with exit code 2 and open a labeled issue naming what was not understood. Never guess.

human-required The label meaning a person must act before automation can continue. Its *removal* is the signal that permits a retry.

Image lock A ConfigMap in the target environment recording which image each service runs. The deployment lane writes to it under compare-and-set; GitOps reconciles from it. A lock write is the whole deploy.

Meta commit One synthetic conventional commit added per synchronization, summarizing the upstream range, whose type is derived by a fixed rule so that version computation stays deterministic without rewriting upstream history.

Mirror mode Customer-tier synchronization behavior, selected by `SYNC_MODE=mirror`, taking the upstream tip verbatim through the same commit plumbing while the filter engine never runs.

OSDU The Open Subsurface Data Universe: an open standard and reference implementation for energy industry data, developed under the OSDU Forum.

Provider model OSDU's structure separating shared service logic from swappable cloud-specific implementations, conventionally as `<svc>-core` alongside `provider/<svc>-azure` and siblings.

Ruleset Branch protection expressed as checked-in JSON and reconciled onto the repository weekly, rather than configured by hand.

Seeding The one-time population of the fork-owned Azure provider and test trees at initialization, taken from the newest upstream revision that still contains them.

Service provider interface (SPI) OSDU's pluggability seam between shared logic and cloud-specific implementation. Also, confusingly, the name of two Azure projects built around it.

SPI Stack `osdu-spi-stack`, the separate project that provisions Azure infrastructure and runs a deployed OSDU environment. The consumer of this system's images and the lender of the environment its lane borrows.

Suite A named group of tests declared in a descriptor, with its own bindings and timeout, packaged in the acceptance image.

SYNC_MODE The repository variable selecting the fork tier. Unset means filter mode; `mirror` means the customer tier. The tier cannot be inferred from the tree, which is why the variable exists.

Template repository `osdu-spi` itself. It owns the workflow logic; each fork owns its configuration data.

Template sync The daily workflow that opens a pull request in each fork when the template has moved, so one fix propagates everywhere without hand edits.

Tier Which kind of fork a repository is. The first tier is a Microsoft-maintained service repository filtered from the community project; the customer tier is a true GitHub fork mirroring a service repository.

Tracking issue An issue used as durable workflow state and as the audit trail for a process spanning hours or days. Carries the hidden marker recording the active upstream SHA.

Trust boundary The rule that credential-bearing jobs run only where the code being built could not have been supplied by an untrusted party. Enforced as a guard clause on each sensitive job.

Upstream-Sha A commit trailer recording the exact upstream commit a generated tree was derived from.

Version stamping The cascade's update of fork-owned Maven POM versions so they stay consistent with upstream-derived coordinates that now move independently.

Appendix A: The Decision Register

Thirty-eight standing records, each stated as its ruling. Numbers 017, 032, and 034 are retired and intentionally unused.

#	Title	Ruling
001	Three-Branch Fork Management	Three long-lived branches — <code>main</code> , <code>fork_upstream</code> , <code>fork_integration</code> — so conflict resolution happens away from production.
002	GitHub Actions Automation	GitHub Actions is the automation substrate; the system is designed around GitHub event semantics.
003	Template Repository Pattern	Forks self-configure from a template that owns the logic while each fork owns its data.
004	Release Please Versioning	Versions and changelogs are computed from conventional commits rather than maintained by hand.
005	Automated Conflict Management	Conflicts are isolated and resolved in the integration branch; upstream never lands on <code>main</code> directly.
006	Two-Workflow Initialization	Initialization splits into two workflows so user input and repository setup fail separately.
007	Initialization Bootstrap	Logic required in a fork's first commit lives in local actions, outside the path that would deliver its own fix.
008	Centralized Label Management	Labels are defined in one file and are part of the system contract, not per-workflow decoration.
009	Asymmetric Cascade Review	Review scrutiny differs by stage; symmetric review on every hop was rejected as slower and noisier.
010	YAML-Safe Shell Scripting	Shell logic is written to be YAML-safe, which pushes substantial logic out of workflow files.
011	Configuration-Driven Template Sync	Template synchronization is driven by configuration files rather than per-fork hardcoding.

#	Title	Ruling
012	Template Update Propagation	Template changes reach forks as reviewable pull requests; one fix propagates everywhere.
013	Reusable Actions for PR Creation	Pull request creation is factored into reusable composite actions instead of duplicated inline.
014	AI-Enhanced Development Workflow	Retired. The AI path crashed on every invocation; descriptions are now deterministic and classification is a rule.
015	Template-Workflows Separation	<code>.github/workflows/</code> is template-only; <code>.github/template-workflows/</code> is what forks receive.
016	Initialization Security Handling	Initialization treats credentials and permissions explicitly rather than as content copying.
018	Fork-Resources Staging	Fork-specific resources are staged before deployment rather than applied in a single step.
019	Cascade Monitor	A six-hourly sweeper detects missed cascades, retries repaired ones, and escalates stale conflicts.
020	Human-Required Label	<code>human-required</code> is the manual-intervention signal; assignment to people failed on orgs, renames, and API errors.
021	Pull Request Target Trigger	Cascade triggers use <code>pull_request_target</code> so the workflow runs from the target branch, never the PR branch.
022	Issue Lifecycle Tracking	Tracking issues hold workflow state and serve as the audit trail for multi-day processes.
023	Meta Commit Strategy	One synthetic commit per sync, typed by a fixed rule; upstream history is never rewritten.
024	Sync Duplicate Prevention	Sync state is durable, label-driven, and fails closed; exactly one pull request per upstream state.
025	Java/Maven Build Architecture	Java 17, Maven, and JaCoCo through reusable actions, against the community Maven repository.
026	Dependabot Strategy	Split by ownership: the template scans its own dependencies; forks scan Maven only, patch-level only.

#	Title	Ruling
027	Documentation Generation	Documentation is Markdown built by MkDocs Material and published from the template only.
028	Workflow Script Extraction	Substantial bash moves into scripts wrapped by composite actions, organized by lifecycle.
029	GitHub App Authentication	All elevated automation uses GitHub App installation tokens; personal access tokens are excluded.
030	CodeQL Summary Job	A job named exactly <code>CodeQL</code> always reports, and the <code>code_scanning</code> ruleset requirement is removed.
031	Template Sync Duplicate Prevention	Template sync reuses its existing labeled branch and pull request rather than opening a new one daily.
033	GHCR as Service Image Registry	Public GHCR is the registry; pulls are anonymous, and a visibility flip is a known unfixable hazard.
035	Azure-Only Maven Profile	CI builds <code>core,azure</code> by default with a per-repository override; a bare <code>-P</code> is never emitted.
036	Workflow Trust Boundaries	Credential-bearing jobs run only in trusted contexts; the guard clause is the policy, not review.
037	Engineering System Owns the Dockerfile	One canonical <code>build/Dockerfile</code> packages the Maven-built JAR; service-owned Dockerfiles are unused.
038	Upstream Filter and Azure Seeding	<code>fork_upstream</code> is generated by a filter, not merged; Azure source is seeded once and fork-owned forever.
039	Customer-Tier Mirror Sync	A second tier mirrors the finished product verbatim, selected by a variable because the tree cannot say.
040	Descriptor Acceptance Contract	Each fork owns a declarative descriptor; the template owns the resolver that joins it to runtime facts.
041	Borrow, Prove, Restore Lane	One credentialed job borrows the environment for its image, proves every declared suite, and always restores.

Appendix B: Workflow Reference

The template's own workflows

Run only in `osdu-spi`. Never deployed to a fork.

File	Name	Purpose
<code>init.yml</code>	Initialize Fork	Detects a new fork, creates labels, opens the initialization issue
<code>init-complete.yml</code>	Initialize Complete	Reads the issue reply and configures the fork end to end
<code>dev-ci.yml</code>	Template Development CI	The template's own build and test harness
<code>dev-release.yml</code>	Release Management	The template's own release automation
<code>docs.yml</code>	Deploy Documentation	Builds and publishes the documentation site
<code>codeql.yml</code>	CodeQL Analysis	Security analysis of the template
<code>scorecard.yml</code>	OpenSSF Scorecard	Supply-chain posture scoring

Workflows deployed to every fork

Delivered from `.github/template-workflows/` at initialization and kept current by template sync.

File	Name	Trigger
<code>sync.yml</code>	Sync Upstream	Daily, and manual dispatch
<code>cascade.yml</code>	Cascade Integration	Manual dispatch, and from the monitor
<code>cascade-monitor.yml</code>	Cascade Monitor	Every six hours, on sync PR close, and manual
<code>validate.yml</code>	Validation	Pull requests and pushes to the three branches
<code>build.yml</code>	Build and Test	Pushes to ordinary feature branches only
<code>release.yml</code>	Release Management	Pushes to <code>main</code> , excluding <code>.github</code> -only changes
<code>integration-cleanup.yml</code>	Integration Branch Cleanup	A merged release pull request on <code>main</code>
<code>sync-template.yml</code>	Sync Template	Daily
<code>adopt-fork.yml</code>	Adopt Fork	Manual dispatch, customer tier

File	Name	Trigger
settings-apply.yml	Settings Apply	Weekly, and manual
dependabot-validation.yml	Dependabot Validation	Dependabot pull requests
ghcr-retention.yml	GHCR Retention	Weekly
codeql.yml	CodeQL Analysis	Weekly and on code changes
copilot-setup-steps.yml	Copilot Setup Steps	Agent environment preparation

Schedules

Cron (UTC)	What runs
0 0 * * *	Upstream synchronization
0 8 * * *	Template synchronization
0 */6 * * *	Cascade monitor
0 4 * * 1	Settings apply
0 5 * * 1	Registry retention
0 6 * * 1	CodeQL analysis
30 1 * * 0	OpenSSF Scorecard

Labels

Defined centrally in `.github/labels.json` and applied consistently by every workflow.

Lifecycle: `initialization`, `upstream-sync`, `sync-update`, `sync-failed`, `template-sync`, `template-update`, `template-sync-failed`, `release-tracking`, `production-ready`

Cascade state: `cascade-active`, `cascade-blocked`, `cascade-ready`, `cascade-failed`, `cascade-escalated`, `upstream-held`, `integration-blocked`, `validated`, `validation-failed`

Intervention: `human-required`, `needs-resolution`, `conflict`, `manual-review-required`, `escalation`, `high-priority`, `emergency`, `rollback`

Other: `build-failed`, `dependencies`, `copilot`

Sources

Everything in this document derives from the `Azure/osdu-spi` repository:

- `README.md`, `CONTRIBUTING.md`, and `CLAUDE.md`
- `doc/src/index.md` and `doc/src/concepts.md`
- `doc/src/architecture/` — the overview, the three-branch strategy, the workflow system, fork tiers, and the deploy and test design
- `doc/src/workflows/` — adoption, initialization, synchronization, cascade, validation, build, and release
- `doc/src/runbooks/` — the fork lifecycle and the service descriptor
- `doc/src/adr/` — the thirty-eight-record decision register, its index, and the companion learnings file
- `.github/` — the workflows, template workflows, composite actions, local actions, rulesets, and label definitions, read directly to verify every exact string cited here

The repository is licensed under Apache 2.0.